

Quantization for Modern AI Systems

From INT8 and LoRA to KV-Cache Compression and TurboQuant

Pawan K Jha

<https://pawankjha.substack.com/>

pawan.jha5@gmail.com

April 4, 2026

Contents

Introduction	6
I Foundations of Quantization	9
1 Why Quantization Matters in Modern AI Systems	10
1.1 Memory Requirements of Large Models	10
1.2 The Compute–Memory Gap	10
2 Floating-Point Representation in Neural Networks	11
2.1 FP32 Representation	11
2.2 FP16 and BF16	11
2.2.1 FP16 (Half Precision)	11
2.2.2 BF16 (Brain Floating Point)	12
2.3 FP8 and Emerging Formats	12
2.4 Comparison of Floating-Point Formats	12
2.5 Dynamic Range vs Precision Trade-offs	13
3 Mixed Precision Training	14
3.1 How AMP Works	14
3.1.1 Loss Scaling	14
3.2 Typical AMP Workflow	14
3.3 Benefits of AMP	15
4 From Floating Point to Quantization	16
4.1 Scalar Quantization Fundamentals	16
4.2 Choosing the Scale Factor	16
4.3 Numerical Example	16
II Quantization in Deep Learning	18
5 Scalar Quantization	19
5.1 Mathematical Formulation	19
5.2 Example	19
5.3 Quantization Error	19
6 Types of Scalar Quantization	20
6.1 Per-Tensor Quantization	20
6.2 Per-Channel Quantization	20
6.3 Symmetric vs Asymmetric Quantization	20
6.3.1 Symmetric Quantization	20
6.3.2 Asymmetric Quantization	20
7 Post-Training Quantization (PTQ)	21
7.1 Pipeline	21
7.2 Calibration	21
7.3 Common PTQ Methods	21
7.3.1 SmoothQuant	21

8	Modern LLM Quantization Methods	23
8.1	GPTQ (Gradient Post-Training Quantization)	23
8.2	AWQ (Activation-Aware Quantization)	23
8.3	QLoRA	23
8.4	GGUF and llama.cpp Quantization	24
8.5	Comparison of LLM Quantization Methods	24
9	Quantization-Aware Training (QAT)	25
9.1	Forward Pass	25
9.2	Backward Pass	25
9.3	Benefits of QAT	25
9.4	Limitations of QAT	25
9.5	QAT vs PTQ Comparison	25
III	Quantization for Large Language Models	26
10	Structure of Transformer Models	27
10.1	Attention Computation	27
10.2	Large Weight Matrices	27
11	The KV Cache Bottleneck	29
11.1	KV Cache Memory Formula	29
11.2	Example: Large LLM	29
11.3	KV Cache Growth Across Context Lengths	30
12	Why KV Cache Compression Is Critical	31
12.1	Benefits of KV Cache Compression	31
12.2	Current Approaches	31
IV	KV Cache Optimization	32
13	KV Cache Growth in Transformer Inference	33
13.1	Attention Computation	33
13.2	KV Cache Structure	33
13.3	Total KV Cache Memory	33
13.4	Example: Large Language Model	34
14	Why KV Cache Becomes the Main Bottleneck	35
14.1	Major Issues	35
14.1.1	Memory Consumption	35
14.1.2	GPU Memory Fragmentation	35
14.1.3	Reduced Batch Size	35
14.1.4	Memory Bandwidth Pressure	35
14.2	KV Bottleneck Comparison	35
15	System-Level KV Cache Optimizations	36
15.1	Paged Attention (vLLM)	36
15.1.1	Traditional vs Paged KV Cache	36
15.1.2	Advantages of Paged Attention	36
15.2	Continuous Batching	36
15.2.1	Benefits	36

15.3 Comparison of System-Level Techniques	37
16 Architectural KV Optimizations	38
16.1 Multi-Head Attention (MHA)	38
16.2 Grouped Query Attention (GQA)	38
16.2.1 KV Memory Comparison	38
16.2.2 Example	39
16.3 Multi-Query Attention (MQA)	39
16.3.1 Benefits	39
16.3.2 Trade-off	39
16.4 Architectural Trade-offs	39
17 Limits of Current KV Optimization Techniques	40
17.1 Precision Limits	40
17.2 The Gap	40
V Vector Quantization	41
18 Why Vector Quantization	42
18.1 Limitations of Scalar Quantization	42
18.2 The Vector Quantization Principle	42
18.3 When Vector Quantization Excels	42
19 Vector Quantization Basics	43
19.1 Memory Savings	43
19.2 Codebook Learning	43
19.3 Encoding and Decoding	44
20 Product Quantization (PQ)	45
20.1 Key Idea	45
20.2 PQ Algorithm	45
20.3 Example	45
20.4 PQ Representation	45
20.5 Reconstruction	45
20.6 Advantages of PQ	46
20.7 Where PQ Is Used	46
20.8 PQ Parameters	46
20.9 Memory Analysis	46
21 Residual Quantization (RQ)	47
21.1 Basic Idea	47
21.2 Algorithm	47
21.3 Step-by-Step Example	47
21.4 Advantages of Residual Quantization	48
21.5 RQ vs PQ: Memory Comparison	48
22 Product Quantization vs Residual Quantization	49
22.1 Comparison Table	49
22.2 When to Use Each	49
22.3 Combined Approaches	49

VI	TurboQuant	50
23	TurboQuant Algorithm	51
23.1	Introduction	51
23.1.1	A New Direction: Vector Quantization for KV Cache	51
23.2	Core Idea	52
23.2.1	Random Rotation	52
23.2.2	Coarse Scalar Quantization	52
23.2.3	Residual Computation	52
23.2.4	Random Projection Sketch	52
23.2.5	Sign Compression	52
23.2.6	Reconstruction	52
23.3	Numerical Example	52
23.3.1	Setup	52
23.3.2	Step 1: Random Rotation	52
23.3.3	Step 2: Coarse Scalar Quantization	53
23.3.4	Step 3: Residual Computation	53
23.3.5	Step 4: Random Projection Sketch	53
23.3.6	Step 5: Sign Compression	53
23.3.7	Step 6: Storage	54
23.3.8	Step 7: Reconstruction	54
23.3.9	Results	55
VII	Quantization in Production ML Systems	56
24	Quantization in Modern Inference Engines	57
24.1	Quantization in vLLM	57
24.1.1	vLLM Inference Pipeline	57
24.1.2	Weight Quantization in vLLM	57
24.1.3	KV Cache Optimization in vLLM	58
24.2	Quantization in TensorRT-LLM	58
24.2.1	TensorRT-LLM Deployment Pipeline	58
24.2.2	INT8 and FP8 Optimization	58
24.3	Comparison of Inference Engines	59
25	Hardware Support for Quantization	60
25.1	GPU Tensor Cores	60
25.2	Tensor Core Throughput Comparison	60
25.3	INT8 Inference Kernels	60
25.3.1	Advantages	61
25.3.2	Applications	61
25.4	FP8 Training Hardware	61
25.4.1	Benefits of FP8	61
25.5	Quantization Support Across Hardware	61
26	Trade-offs in Quantization	62
26.1	Accuracy vs Compression	62
26.2	Latency vs Decompression Overhead	62
26.3	Calibration Complexity	62
26.4	Trade-off Summary	63

27 Choosing a Quantization Strategy	64
27.1 Decision Framework	64
27.2 Combined Strategies	64
 VIII Future Directions in Quantization	 65
28 Adaptive Precision	66
28.1 Layer-Wise Precision	66
28.2 Token-Level Precision	66
28.3 Benefits of Adaptive Precision	66
29 Learned Quantization	67
29.1 Core Idea	67
29.2 Differentiable Quantization	67
29.3 Learned Parameters	67
29.4 Benefits of Learned Quantization	67
30 Token-Level KV Compression	68
30.1 Motivation	68
30.2 Token-Level Strategies	68
30.3 Token-Level KV Compression Example	68
30.4 Expected Benefits	68
31 Hybrid Compression Methods	69
31.1 Example: Low-Rank + Quantization	69
31.2 Combined Approach Benefits	69
31.3 Other Hybrid Combinations	69
32 Hardware–Algorithm Co-Design	70
32.1 Examples of Co-Design	70
32.2 Benefits of Co-Design	70
32.3 The Co-Design Feedback Loop	70
33 Toward Efficient AI Systems	71
33.1 The Efficiency Stack	71
33.2 Expected Outcomes	71
34 Final Thoughts	72
34.1 The Journey	72
34.2 Key Takeaways	72
34.3 A Look Ahead	72

Introduction

Over the last decade, advances in deep learning have been driven largely by **scale**—larger datasets, larger models, and more compute. Modern AI systems routinely deploy models with **billions or even hundreds of billions of parameters**, such as large language models (LLMs) used for chatbots, search, code generation, and reasoning.

However, this scale comes with a fundamental constraint: **hardware memory and bandwidth**.

Even moderately sized models require enormous memory footprints. For example, a 70-billion parameter transformer stored in FP16 precision requires roughly:

$$70B \text{ parameters} \times 2 \text{ bytes} \approx 140 \text{ GB}$$

This memory requirement alone exceeds the capacity of a single GPU. In practice, production deployments must also account for:

- activation memory during inference
- intermediate tensors during computation;
- **key-value (KV) caches** used for autoregressive attention
- embedding storage for retrieval systems

As sequence lengths increase, particularly in modern long-context models supporting **32k, 128k, or even million-token contexts**, the **KV cache quickly becomes one of the dominant memory bottlenecks** in transformer inference.

In large-scale deployments, memory, not computation, often becomes the limiting factor.

The Role of Quantization

To address these challenges, modern AI systems rely heavily on **quantization**, a technique that reduces the numerical precision used to represent tensors such as weights, activations, and embeddings.

Instead of storing values in 32-bit floating point format, quantization represents them using fewer bits—such as 16-bit, 8-bit, or even 4-bit representations—dramatically reducing memory usage and improving hardware efficiency.

Mathematically, quantization maps continuous values to a smaller discrete set:

$$q = \text{round} \left(\frac{x}{s} \right)$$

where:

- x is the original value
- s is a scaling factor
- q is the quantized integer representation

The original value can then be approximately reconstructed as:

$$\hat{x} = s \cdot q$$

Despite this approximation, modern quantization methods can often preserve model accuracy surprisingly well.

Quantization Across the AI Stack

Today, quantization is applied across multiple layers of modern AI systems:

Component	Purpose
Model weights	reduce model size and memory bandwidth
Activations	reduce intermediate tensor memory
KV cache	reduce memory during long-context inference
Embeddings	compress vectors in large retrieval systems

In large language models, weight quantization methods such as **GPTQ**, **AWQ**, and **QLoRA** allow models to run with **4-bit or 8-bit precision** while maintaining high accuracy.

Similarly, vector search systems rely heavily on **vector quantization techniques** such as **Product Quantization (PQ)** to compress high-dimensional embeddings.

However, as models continue to scale and support increasingly long contexts, another challenge has emerged.

The KV Cache Problem

Transformer inference relies on the **attention mechanism**, where each token attends to previously generated tokens using stored key and value vectors.

For a sequence of length T , each layer stores:

$$K \in \mathbb{R}^{T \times d}, \quad V \in \mathbb{R}^{T \times d}$$

These vectors accumulate during generation and must remain accessible for future attention computations.

As a result, KV cache memory grows linearly with sequence length.

For large models with dozens of layers and long context windows, KV cache storage can exceed the memory required for model weights themselves.

Even aggressive scalar quantization (such as INT8 or FP8) only partially mitigates this problem.

This has led to a growing research focus on more advanced compression methods for KV caches.

A New Direction: Vector-Based KV Compression

Recent work explores whether KV cache vectors can be compressed more aggressively using **vector quantization techniques** that capture correlations between dimensions rather than quantizing each element independently.

One notable example is **TurboQuant**, a recent technique proposed by researchers at Google.

Rather than applying simple scalar quantization to each element, TurboQuant combines several ideas:

- random rotations to redistribute vector information
- coarse scalar quantization
- randomized projection sketches of residual errors
- sign compression for efficient storage

This hybrid approach allows KV vectors to be compressed to as little as **3–4 bits per dimension** while maintaining high attention accuracy.

The key insight is that transformer KV vectors exhibit structured redundancy, making them amenable to more sophisticated compression strategies than traditional quantization methods.

What This Book Covers

This book explores quantization from first principles to cutting-edge techniques used in modern AI infrastructure.

We will begin with the mathematical foundations of scalar quantization, then explore how quantization is applied in deep learning systems, including:

- weight and activation quantization
- mixed precision training
- quantization for large language models
- low-rank methods such as LoRA and QLoRA

Next, we will examine **KV cache optimization techniques** used in transformer inference, including architectural improvements such as grouped query attention and paged attention.

We will then dive into **vector quantization algorithms**, including:

- classical vector quantization
- product quantization
- residual quantization
- optimized product quantization

Finally, we will analyze **TurboQuant**, walking through the algorithm step-by-step with a concrete example and discussing its implications for the future of large-scale AI systems.

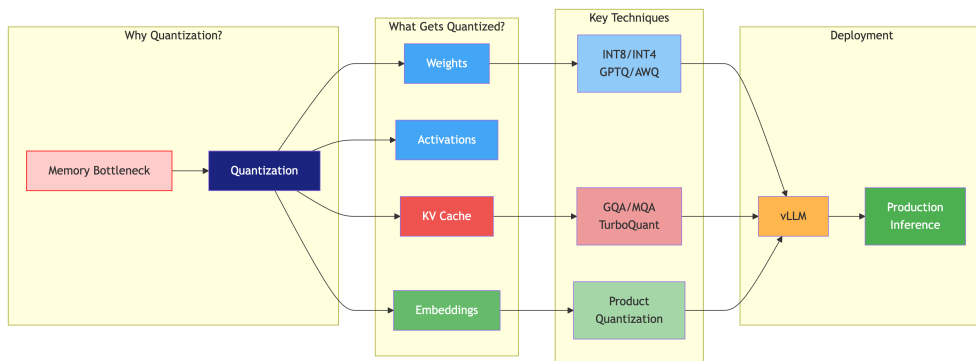


Figure 1: Transformer block showing where quantization is applied (Red: Weight Quantization, Green: Activation Quantization, Blue: KV Cache Quantization)

Part I

Foundations of Quantization

1 Why Quantization Matters in Modern AI Systems

Deep learning progress has largely been driven by scaling laws: larger models trained on more data with more compute tend to perform better.

However, this growth creates severe hardware bottlenecks.

1.1 Memory Requirements of Large Models

Consider a transformer model with 70 billion parameters.

If stored in FP16 precision:

$$70B \text{ parameters} \times 2 \text{ bytes} \approx 140 \text{ GB}$$

This exceeds the memory of a single modern GPU. In practice, additional memory is also required for:

- activations
- intermediate tensors
- optimizer states (during training)
- KV cache (during inference)

Note: During inference, model weights and KV cache dominate memory. During training, optimizer states — typically stored in FP32 — add significant overhead, often 2× to 4× the weight memory.

1.2 The Compute–Memory Gap

Modern GPUs have extremely high compute throughput but comparatively limited memory bandwidth. This imbalance means that many deep learning workloads are **memory-bound** rather than compute-bound.

Quantization helps address this problem by reducing:

- memory footprint
- memory bandwidth requirements
- tensor transfer costs

This enables:

- larger batch sizes
- larger models
- higher inference throughput

2 Floating-Point Representation in Neural Networks

Neural networks typically store numbers using floating-point formats. A floating-point number consists of three components:

- **sign** — determines positive or negative
- **exponent** — controls magnitude (dynamic range)
- **mantissa (fraction)** — controls numerical precision

The numerical value is computed as:

$$\text{value} = (-1)^{\text{sign}} \times (1 + \text{mantissa}) \times 2^{(\text{exponent} - \text{bias})}$$

Different formats allocate different numbers of bits to these components, creating trade-offs between **precision**, **dynamic range**, and **memory usage**.

2.1 FP32 Representation

The standard floating-point format used in early deep learning systems is **FP32** (single precision).

Component	Bits
Sign	1
Exponent	8
Mantissa	23

Total: 32 bits (4 bytes)

FP32 offers high numerical precision and stability, making it suitable for training neural networks. However, it is expensive in terms of memory and compute.

2.2 FP16 and BF16

To improve efficiency, modern deep learning systems often use 16-bit floating-point formats.

2.2.1 FP16 (Half Precision)

Component	Bits
Sign	1
Exponent	5
Mantissa	10

Advantages:

- Reduces memory by 2× compared to FP32
- Faster GPU tensor operations

Disadvantage: Limited dynamic range, which can cause numerical instability during training.

2.2.2 BF16 (Brain Floating Point)

BF16 keeps the same exponent size as FP32 while reducing mantissa precision.

Component	Bits
Sign	1
Exponent	8
Mantissa	7

Advantages:

- Same dynamic range as FP32 (stable training)
- Reduced memory footprint

BF16 is widely used for training large transformer models.

2.3 FP8 and Emerging Formats

Recent hardware architectures now support **FP8**, which reduces precision even further.

FP8 comes in two main variants:

Variant	Exponent Bits	Mantissa Bits	Best For
E4M3	4	3	Higher precision, limited dynamic range
E5M2	5	2	Higher dynamic range, lower precision

Benefits:

- 75% memory reduction compared to FP16
- Faster tensor core operations
- Enables larger batch sizes

FP8 training is supported in modern GPUs such as NVIDIA H100.

2.4 Comparison of Floating-Point Formats

Format	Total Bits	Exp Bits	Mant Bits	Dynamic Range	Typical Use
FP32	32	8	23	$\sim 10^{-38}$ to 10^{38}	Training baseline
FP16	16	5	10	$\sim 6 \times 10^{-8}$ to 65504	Inference, some training
BF16	16	8	7	$\sim 10^{-38}$ to 10^{38}	Large model training
FP8 (E4M3)	8	4	3	Limited	Inference, emerging
FP8 (E5M2)	8	5	2	Higher	Inference, gradients

2.5 Dynamic Range vs Precision Trade-offs

Reducing numerical precision introduces two key trade-offs:

- **Dynamic Range:** The exponent determines the range of representable values. A larger dynamic range allows the system to represent very small gradients and very large activations without overflow or underflow.
- **Precision:** The mantissa determines how precisely numbers can be represented. Lower mantissa precision introduces quantization noise.

Example: Consider the value 0.123456

Format	Approximate Value	Relative Error
FP32	0.123456	—
FP16	0.1235	$\sim 0.04\%$
FP8 (E4M3)	0.125	$\sim 1.2\%$

The difference between the original and stored value is called **quantization error**. Neural networks are often surprisingly robust to this error, especially when quantization is applied thoughtfully.

3 Mixed Precision Training

Most modern training pipelines use **Automatic Mixed Precision (AMP)**, which dynamically combines different numerical formats during training.

3.1 How AMP Works

The key insight is that while forward and backward passes can use lower precision, certain operations require higher precision to maintain stability:

Component	Precision	Rationale
Master weights	FP32	Preserve training stability
Forward pass	FP16/BF16	Reduce memory and compute
Backward pass	FP16/BF16	Reduce memory and compute
Gradients	FP16/BF16	Temporary storage
Gradient accumulation	FP32	Prevent underflow
Weight updates	FP32	Apply updates to master weights

3.1.1 Loss Scaling

A critical technique in AMP is **loss scaling**. When using FP16 for gradients, very small gradient values can underflow to zero, especially in deep networks.

Loss scaling multiplies the loss by a constant s before backpropagation:

$$L_{\text{scaled}} = s \times L$$

After computing scaled gradients ∇_{scaled} , they are divided by s before weight updates:

$$\nabla = \frac{\nabla_{\text{scaled}}}{s}$$

This preserves small gradient values that would otherwise underflow in FP16. Modern frameworks like PyTorch and TensorFlow implement dynamic loss scaling that automatically adjusts the scale factor during training.

3.2 Typical AMP Workflow

Listing 3.1: AMP Workflow

```
Master weights (FP32)
  |
  v
Cast to FP16/BF16
  |
  v
Forward pass (FP16/BF16)
  |
  v
Loss computation (FP32)
  |
  v
```

```
Loss scaling: L_scaled = s * L
|
v
Backward pass (FP16/BF16)
|
v
Gradients -> accumulate in FP32
|
v
Unscale gradients: grad = grad / s
|
v
Update master weights (FP32)
```

3.3 Benefits of AMP

- **Faster training:** Up to $2\times$ speedup on compatible hardware
- **Lower memory usage:** Activations and gradients use half the memory
- **Stable gradient updates:** FP32 accumulation prevents underflow

AMP does not change the model architecture. Instead, it changes the numerical precision used during computation, making it a lightweight optimization that delivers significant benefits.

4 From Floating Point to Quantization

Floating-point compression methods such as FP16 and FP8 reduce precision but still represent numbers in floating-point format. **Quantization** goes further by converting numbers into discrete integer representations.

4.1 Scalar Quantization Fundamentals

The simplest form of quantization is **uniform scalar quantization**, defined as:

$$q = \text{round}\left(\frac{x}{s}\right)$$

Reconstruction is then performed using:

$$\hat{x} = s \cdot q$$

Where:

- x = original floating-point value
- s = scale factor
- q = integer representation
- $\hat{x}$ = reconstructed value

This transformation drastically reduces memory requirements because q can be stored using far fewer bits than the original x .

4.2 Choosing the Scale Factor

The scale factor s determines quantization resolution. Common approaches include:

Method	Description	Use Case
Max-based	$s = \max(x)/(2^{b-1} - 1)$ for symmetric quantization	Simple, preserves full range
Percentile-based	Clip outliers at 99.9th percentile to avoid distortion	Handles extreme outliers
Min-max	$s = (\max(x) - \min(x))/(2^b - 1)$ for asymmetric	Better for non-symmetric distributions
Learned	Optimize s during calibration to minimize reconstruction error	Highest accuracy

4.3 Numerical Example

Consider the vector:

$$x = [0.83, -0.31, 0.27, 0.11]$$

Choose scale factor $s = 0.25$.

Quantization:

$$q = \text{round}(x/s) = [3, -1, 1, 0]$$

Reconstruction:

$$\hat{x} = s \cdot q = [0.75, -0.25, 0.25, 0]$$

Quantization error:

Original	Quantized	Reconstructed	Error
0.83	3	0.75	0.08
-0.31	-1	-0.25	-0.06
0.27	1	0.25	0.02
0.11	0	0	0.11

Although some information is lost, neural networks are surprisingly robust to these approximations. The key is choosing a scale factor that preserves the information most important for model accuracy.

Part II

Quantization in Deep Learning

5 Scalar Quantization

The simplest and most widely used quantization method is **scalar quantization**, where each individual value is quantized independently.

The goal is to map floating-point values to a small set of discrete integer levels.

5.1 Mathematical Formulation

Let a floating-point tensor be $x \in \mathbb{R}$. We define a scale factor s and compute the quantized integer value:

$$q = \text{round}\left(\frac{x}{s}\right)$$

The original value can be approximately reconstructed as:

$$\hat{x} = s \cdot q$$

5.2 Example

Suppose we have a tensor:

$$x = [0.83, -0.31, 0.27, 0.11]$$

Choose scale $s = 0.25$.

Quantization:

$$q = \text{round}(x/s) = [3, -1, 1, 0]$$

Reconstruction:

$$\hat{x} = s \cdot q = [0.75, -0.25, 0.25, 0]$$

Although the reconstructed values are not exact, the error is often small enough that model accuracy remains unchanged.

5.3 Quantization Error

The difference between original and reconstructed values is:

$$\epsilon = x - \hat{x}$$

Deep neural networks tend to tolerate moderate quantization noise because:

- many neurons contribute to outputs
- gradients average across parameters
- transformers have redundancy in representations

This robustness makes quantization highly effective in practice.

6 Types of Scalar Quantization

In real systems, quantization is applied in different ways depending on the structure of the tensor.

6.1 Per-Tensor Quantization

A single scale factor is used for the entire tensor.

$$q = \text{round} \left(\frac{x}{s} \right)$$

Advantages:

- simple implementation
- minimal metadata

Disadvantages:

- may perform poorly if tensor values vary widely

6.2 Per-Channel Quantization

Each channel (or dimension) uses a separate scale factor.

$$q_i = \text{round} \left(\frac{x_i}{s_i} \right)$$

Advantages:

- better accuracy
- adapts to different distributions

This is commonly used for:

- convolution layers
- transformer weight matrices

6.3 Symmetric vs Asymmetric Quantization

6.3.1 Symmetric Quantization

Zero is centered in the quantization range.

$$q = \text{round} \left(\frac{x}{s} \right)$$

Used when tensor distributions are roughly symmetric around zero.

6.3.2 Asymmetric Quantization

Includes a zero-point offset:

$$q = \text{round} \left(\frac{x}{s} \right) + z$$

Where z is the zero-point offset.

This allows the quantization range to better match non-centered distributions (e.g., ReLU activations).

7 Post-Training Quantization (PTQ)

Post-training quantization converts a trained model to lower precision without retraining.

7.1 Pipeline

Listing 7.1: PTQ Pipeline

```
trained model
  |
  v
collect calibration data
  |
  v
estimate scale factors
  |
  v
convert weights to INT8 / INT4
```

PTQ is widely used because it is:

- fast
- simple
- inexpensive

7.2 Calibration

PTQ requires a small calibration dataset to estimate tensor ranges.

Example procedure:

1. run inference on sample data
2. record activation ranges
3. compute scale factors

This ensures quantization does not cause large clipping errors.

7.3 Common PTQ Methods

Method	Description
Min-Max	Uses full range of values: $s = (\max(x) - \min(x)) / (2^b - 1)$
Percentile	Clips outliers at 99.9th percentile to avoid distortion
Entropy-based	Minimizes KL divergence between original and quantized distributions

7.3.1 SmoothQuant

SmoothQuant (ICML 2023) is a modern PTQ technique designed specifically for transformers.

The key insight is that activation outliers cause quantization difficulty. SmoothQuant addresses this by **migrating quantization difficulty from activations to weights**:

$$Y = (X \cdot \text{diag}(s)^{-1}) \cdot (\text{diag}(s) \cdot W)$$

Where s is a per-channel smoothing factor chosen to balance quantization difficulty between activations and weights.

Benefits:

- Preserves accuracy at INT8 quantization for LLMs
- No retraining required
- Widely used in production deployments

8 Modern LLM Quantization Methods

Large language models introduce unique challenges:

- extremely large weight matrices
- sensitive attention layers
- long context sequences

Several specialized PTQ algorithms have emerged.

8.1 GPTQ (Gradient Post-Training Quantization)

GPTQ minimizes quantization error using approximate second-order optimization.

Idea:

$$\min \|W - W_q\|_H$$

Where H approximates the Hessian of the loss.

Benefits:

- high accuracy at 4-bit precision
- widely used for LLM inference

8.2 AWQ (Activation-Aware Quantization)

AWQ identifies important weights by analyzing activation statistics.

Weights that strongly influence activations receive higher precision.

Advantages:

- preserves accuracy in transformer layers
- effective for instruction-tuned models

8.3 QLoRA

QLoRA combines 4-bit quantization with low-rank adapters.

Process:

Listing 8.1: QLoRA Workflow

```
base model -> quantize to 4-bit
      |
      v
add LoRA adapters
      |
      v
train adapters only
```

Benefits:

- fine-tuning large models with limited GPU memory
- avoids updating full weight matrices

8.4 GGUF and llama.cpp Quantization

For edge and CPU-based inference, the **GGUF** format (GGUF = GGML Universal Format) has become the standard.

Common GGUF quantization types:

Format	Bits per Weight	Description
Q4_0	4.0	Basic 4-bit quantization
Q4_K	4.5	K-means optimized 4-bit
Q5_K	5.5	Higher quality 5-bit
Q8_0	8.0	Near-lossless 8-bit

These formats enable running 7B-70B parameter models on consumer hardware using frameworks like:

- **llama.cpp** — C++ inference engine
- **Ollama** — Easy-to-use packaging
- **LM Studio** — GUI for local models

8.5 Comparison of LLM Quantization Methods

Method	Precision	Training Required	Typical Use
GPTQ	4-bit	None	Inference
AWQ	4-bit	None	Inference
QLoRA	4-bit	Fine-tuning	Parameter-efficient tuning
GGUF	4-8 bit	None	Edge/CPU inference
bitsandbytes	8-bit	None	Quick inference

9 Quantization-Aware Training (QAT)

Post-training quantization works well in many cases, but extreme compression can degrade accuracy.

An alternative approach is **Quantization-Aware Training (QAT)**.

In QAT, quantization effects are simulated during training.

9.1 Forward Pass

Weights are quantized before computation:

$$x_q = \text{quantize}(x)$$

9.2 Backward Pass

Gradients are approximated using the **Straight-Through Estimator (STE)**.

$$\frac{\partial L}{\partial x} \approx \frac{\partial L}{\partial x_q}$$

This allows gradients to propagate through the quantization operation.

9.3 Benefits of QAT

Compared to PTQ, QAT can:

- maintain higher accuracy
- enable more aggressive compression
- adapt model weights to quantization noise

9.4 Limitations of QAT

However, QAT requires retraining the model, which is expensive for large LLMs.

For this reason, most LLM deployments rely on PTQ methods like GPTQ and AWQ.

9.5 QAT vs PTQ Comparison

Aspect	PTQ	QAT
Training required	None	Full or fine-tuning
Accuracy	Good	Better
Compression	Moderate	Aggressive
Compute cost	Low	High
Typical use	Production inference	Research, critical applications

Part III

Quantization for Large Language Models

10 Structure of Transformer Models

Modern LLMs are built using stacks of transformer blocks.

Each block typically contains two main components:

- Multi-Head Self-Attention
- Feed-Forward Network (FFN)

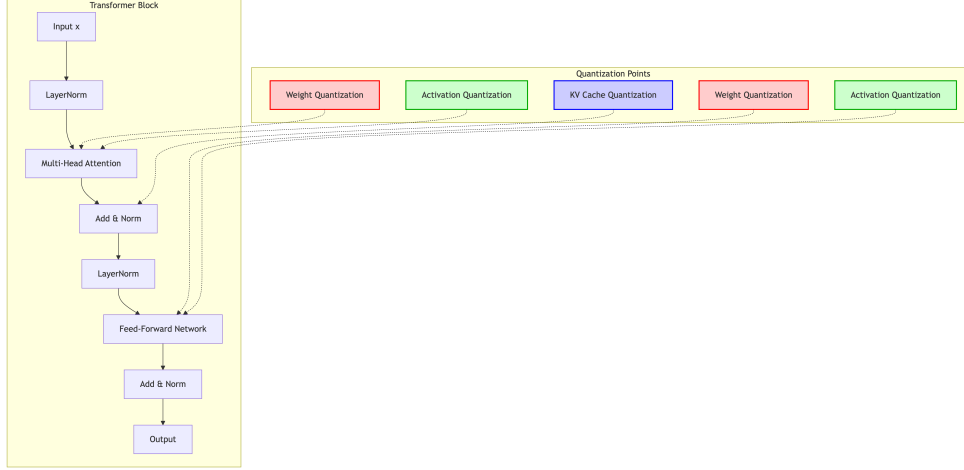


Figure 10.1: Transformer block showing where quantization is applied (Red: Weight Quantization, Green: Activation Quantization, Blue: KV Cache Quantization)

10.1 Attention Computation

Self-attention computes relationships between tokens using query, key, and value projections.

For a hidden state vector x :

$$Q = xW_Q, \quad K = xW_K, \quad V = xW_V$$

The attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Where:

- d is the attention dimension
- W_Q, W_K, W_V are projection matrices

10.2 Large Weight Matrices

Transformer layers contain very large weight matrices.

Typical dimensions:

$$W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$$

Feed-forward layers contain even larger matrices:

$$W_1 \in \mathbb{R}^{d \times 4d}, \quad W_2 \in \mathbb{R}^{4d \times d}$$

For models with thousands of hidden units and dozens of layers, these matrices dominate the parameter count.

11 The KV Cache Bottleneck

During autoregressive inference, LLMs must remember previous tokens.

Instead of recomputing attention for every token, models store intermediate vectors called **Key (K)** and **Value (V)** states.

For a sequence length T , each transformer layer stores:

$$K \in \mathbb{R}^{T \times d}, \quad V \in \mathbb{R}^{T \times d}$$

These matrices grow as new tokens are generated.

11.1 KV Cache Memory Formula

The total KV memory requirement is approximately:

$$\text{Memory} = 2 \times L \times T \times d \times \text{bytes}$$

Where:

- L = number of transformer layers
- T = sequence length
- d = hidden dimension
- bytes = bytes per value (e.g., 2 for FP16)

11.2 Example: Large LLM

Consider a model with:

- Layers (L) = 80
- Hidden dimension (d) = 8192
- Sequence length (T) = 4096
- Precision = FP16 (2 bytes)

KV memory becomes:

$$\begin{aligned} \text{Memory} &\approx 2 \times 80 \times 4096 \times 8192 \times 2 \\ &= 10.7 \text{ GB} \end{aligned}$$

This is **per request**.

For high-throughput serving systems, KV cache can easily consume tens or hundreds of gigabytes.

11.3 KV Cache Growth Across Context Lengths

Sequence Length	KV Cache Size (FP16)	Use Case
1K tokens	~2.7 GB	Short conversation
4K tokens	~10.7 GB	Typical chatbot
32K tokens	~85.6 GB	Long document analysis
128K tokens	~342 GB	Extended reasoning
1M tokens	~2.7 TB	Full book processing

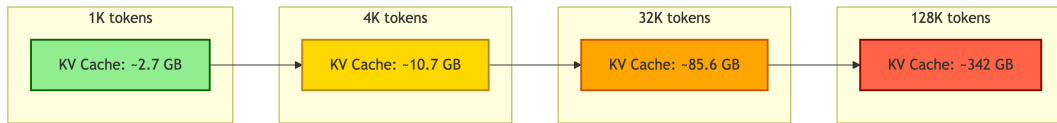


Figure 11.1: KV cache growth with sequence length (70B model, FP16). Memory grows linearly with context length, becoming the dominant bottleneck for long sequences.

12 Why KV Cache Compression Is Critical

While weight quantization reduces model size, KV cache grows dynamically with sequence length.

This means KV memory becomes the dominant bottleneck for:

- long-context models
- chat applications
- multi-user serving systems

12.1 Benefits of KV Cache Compression

Reducing KV memory usage enables:

- **longer context windows** — process more tokens per request
- **higher concurrency** — serve more users simultaneously
- **better GPU utilization** — pack more requests per GPU

12.2 Current Approaches

Existing approaches to KV cache compression include:

- **Scalar quantization:** INT8, INT4 per-element quantization
- **Architectural changes:** Grouped Query Attention (GQA), Multi-Query Attention (MQA)
- **Memory management:** PagedAttention (vLLM)
- **Advanced compression:** Vector quantization, TurboQuant

These techniques form the foundation for efficient LLM deployment and will be explored in the following sections.

Part IV

KV Cache Optimization

13 KV Cache Growth in Transformer Inference

During autoregressive generation, the model produces tokens sequentially.

For each new token:

- the model computes query (Q) for the current token
- previously computed keys (K) and values (V) are reused
- attention is computed against the entire KV cache

13.1 Attention Computation

For token t , attention is computed as:

$$\text{Attention}(Q_t, K_{1:t}, V_{1:t}) = \text{softmax}\left(\frac{Q_t K_{1:t}^T}{\sqrt{d}}\right) V_{1:t}$$

Where:

- Q_t = query vector for current token
- $K_{1:t}$ = keys from all previous tokens
- $V_{1:t}$ = values from all previous tokens

This means the model must store all previous K and V vectors.

13.2 KV Cache Structure

For each transformer layer we store:

$$K \in \mathbb{R}^{T \times d}, \quad V \in \mathbb{R}^{T \times d}$$

Where:

- T = sequence length
- d = hidden dimension

13.3 Total KV Cache Memory

Total memory required for KV cache is approximately:

$$\text{Memory} = 2 \times L \times T \times d \times \text{bytes}$$

Where:

- L = number of transformer layers
- T = sequence length
- d = hidden dimension
- bytes = numerical precision (e.g., FP16 = 2 bytes)

13.4 Example: Large Language Model

Consider a model with:

- Layers (L) = 80
- Hidden dimension (d) = 8192
- Sequence length (T) = 4096
- Precision = FP16 (2 bytes)

KV memory becomes:

$$\begin{aligned}\text{Memory} &\approx 2 \times 80 \times 4096 \times 8192 \times 2 \\ &= 10.7 \text{ GB}\end{aligned}$$

This is **per request**.

For high-throughput serving systems handling thousands of requests, KV cache memory can easily exceed hundreds of gigabytes.

14 Why KV Cache Becomes the Main Bottleneck

Weight quantization can reduce model size significantly.

However, KV cache grows dynamically during inference.

As context windows increase (32K, 128K, or even 1M tokens), KV memory becomes the primary constraint.

14.1 Major Issues

14.1.1 Memory Consumption

Large context windows require enormous KV storage. For 128K tokens with a 70B model, KV cache alone can exceed 300 GB per request.

14.1.2 GPU Memory Fragmentation

Dynamic allocation of KV memory can cause inefficient GPU memory usage. As requests of varying lengths are processed, memory becomes fragmented, reducing effective capacity.

14.1.3 Reduced Batch Size

Large KV cache reduces the number of concurrent requests a GPU can serve. This directly impacts throughput and serving costs.

14.1.4 Memory Bandwidth Pressure

Frequent reads of KV tensors increase memory bandwidth demand. In many inference workloads, memory bandwidth becomes the limiting factor, not compute.

14.2 KV Bottleneck Comparison

Model Component	Size (70B FP16)	Grows with Context?	Per-Request?
Weights	140 GB	No	Shared
Activations	~1-5 GB	No	Per-request
KV Cache	10-300+ GB	Yes	Per-request

15 System-Level KV Cache Optimizations

Several system-level techniques have been developed to improve KV cache efficiency.

These methods focus on memory management rather than numerical compression.

15.1 Paged Attention (vLLM)

One major innovation used in **vLLM** is **Paged Attention**.

Instead of storing KV tensors in large contiguous blocks, memory is organized into fixed-size pages.

15.1.1 Traditional vs Paged KV Cache

Listing 15.1: Traditional Contiguous KV Cache

```
Token1 Token2 Token3 Token4 Token5  
[ contiguous memory allocation ]  
Fragmentation: large gaps between requests
```

Listing 15.2: Paged Attention KV Cache

```
Page1: Token1 Token2  
Page2: Token3 Token4  
Page3: Token5  
  
Pages can be non-contiguous and reused across requests
```

15.1.2 Advantages of Paged Attention

- **Avoids memory fragmentation** — pages are allocated in fixed sizes
- **Enables efficient memory reuse** — pages can be shared across requests
- **Improves multi-request batching** — more requests fit in GPU memory

Paged Attention significantly improves throughput for LLM serving systems, achieving 2× to 4× higher throughput compared to traditional systems.

15.2 Continuous Batching

Many inference engines implement **continuous batching**, where multiple requests share computation.

Instead of processing each request independently, tokens from different requests are processed together.

15.2.1 Benefits

- **Better GPU utilization** — keeps compute units busy
- **Improved throughput** — more tokens processed per second
- **Lower latency variance** — fair scheduling across requests

However, KV cache must still store state for each request separately.

15.3 Comparison of System-Level Techniques

Technique	Primary Benefit	Implementation
Paged Attention	Reduced fragmentation	vLLM, SGLang
Continuous Batching	Higher throughput	TensorRT-LLM, vLLM
Speculative Decoding	Lower latency	Multiple frameworks

16 Architectural KV Optimizations

Beyond system-level improvements, several architectural changes reduce KV storage requirements.

16.1 Multi-Head Attention (MHA)

Standard transformer attention uses multiple independent heads.

Each head maintains its own KV tensors:

$$\text{Memory (MHA)} = 2 \times L \times T \times d_{\text{head}} \times n_{\text{heads}}$$

This multiplies KV memory usage by the number of heads.

16.2 Grouped Query Attention (GQA)

Grouped Query Attention reduces KV storage by sharing keys and values across multiple query heads.

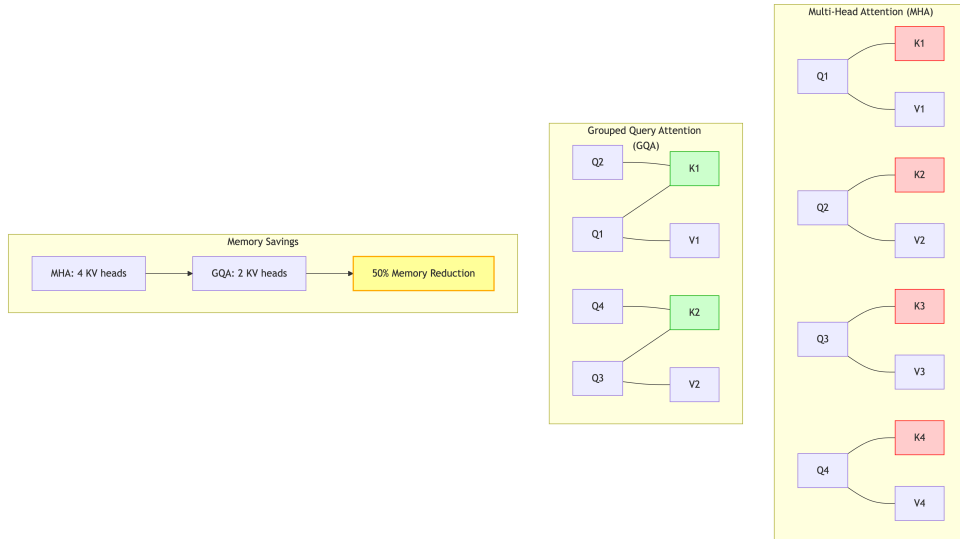


Figure 16.1: Multi-Head Attention (MHA) vs Grouped Query Attention (GQA). GQA shares KV heads across query groups, reducing memory by 50% or more.

16.2.1 KV Memory Comparison

Attention Type	KV Head Count	Relative KV Memory
Multi-Head (MHA)	n_{heads}	$1.0\times$
Grouped Query (GQA)	n_{groups}	$1/n_{\text{groups}}$
Multi-Query (MQA)	1	$1/n_{\text{heads}}$

16.2.2 Example

For a model with 32 heads and GQA with 8 groups:

- MHA: 32 KV heads $\rightarrow$ baseline
- GQA: 8 KV heads $\rightarrow 4\times$ reduction
- MQA: 1 KV head $\rightarrow 32\times$ reduction

16.3 Multi-Query Attention (MQA)

An even more aggressive approach is **Multi-Query Attention**, where all query heads share the same K and V .

16.3.1 Benefits

- Major KV memory reduction ($16\times$ to $32\times$)
- Faster inference due to reduced memory bandwidth

16.3.2 Trade-off

- Slight reduction in model capacity
- May require additional training to maintain quality

16.4 Architectural Trade-offs

Attention Type	KV Memory	Model Quality	Inference Speed
MHA	High	Best	Baseline
GQA	Medium	Good	Faster
MQA	Low	Acceptable	Fastest

17 Limits of Current KV Optimization Techniques

While the methods above improve memory efficiency, they do not fundamentally compress the KV tensors themselves.

The underlying KV vectors are still stored using floating-point precision.

17.1 Precision Limits

Precision	Bytes per Value	Relative to FP16
FP16	2 bytes	1.0×
FP8	1 byte	0.5×
INT8	1 byte	0.5×
INT4	0.5 bytes	0.25×

Even with INT4 compression, KV cache size remains substantial for long contexts.

17.2 The Gap

For a 128K context with a 70B model using GQA:

- With FP16: ~342 GB
- With INT4: ~85 GB

This is still significant memory pressure for multi-user serving.

Part V

Vector Quantization

18 Why Vector Quantization

18.1 Limitations of Scalar Quantization

Scalar quantization compresses each value independently, ignoring relationships between dimensions. In high-dimensional vectors, dimensions are often correlated.

Consider an embedding vector where:

- Dimension 1 represents "positive sentiment"
- Dimension 2 represents "negative sentiment"

These dimensions are strongly correlated. Quantizing them independently wastes this structural information.

18.2 The Vector Quantization Principle

Vector quantization compresses entire vectors by mapping them to prototype vectors in a codebook:

$$x \approx c_i, \quad c_i \in C = \{c_1, c_2, \dots, c_k\}$$

Instead of storing d floating-point values, we store a single index i .

18.3 When Vector Quantization Excels

Vector quantization is particularly effective for:

- **Embeddings** — word, sentence, or document vectors
- **Attention vectors** — K and V in transformers
- **Image patches** — ViT embeddings
- **Search indexes** — FAISS, vector databases

19 Vector Quantization Basics

Consider a vector:

$$x \in \mathbb{R}^d$$

Instead of storing the full vector, vector quantization approximates it using a codebook vector.

Let the codebook be:

$$C = \{c_1, c_2, \dots, c_k\}$$

Where each c_i is a prototype vector (also called a centroid).

The original vector is approximated by the nearest codebook entry:

$$x \approx c_i$$

Rather than storing the full vector x , we store only the index i .

This dramatically reduces memory usage.

19.1 Memory Savings

Suppose:

- vector dimension = 128
- floating-point precision = FP32 (32 bits)

Memory required:

$$128 \times 32 \text{ bits} = 4096 \text{ bits}$$

If we store only a codebook index with 8 bits:

$$8 \text{ bits}$$

Compression ratio:

$$\frac{4096}{8} = 512\times$$

In practice, additional information (codebook storage) is required, but vector quantization still provides very strong compression.

19.2 Codebook Learning

The codebook is typically learned using **k-means clustering** on a representative dataset:

Listing 19.1: Codebook Learning

```
Input: Training vectors  $X = \{x_1, x_2, \dots, x_n\}$ 
Output: Codebook  $C = \{c_1, c_2, \dots, c_k\}$ 

1. Initialize  $k$  centroids
2. Repeat until convergence:
    a. Assign each vector to nearest centroid
    b. Update centroids as mean of assigned vectors
3. Return final centroids
```

19.3 Encoding and Decoding

Encoding: For a vector x , find the nearest codebook entry:

$$i = \arg \min_j \|x - c_j\|_2$$

Decoding: Reconstruct the vector:

$$\hat{x} = c_i$$

20 Product Quantization (PQ)

Product Quantization is one of the most widely used vector compression techniques.

It was introduced to efficiently compress high-dimensional vectors for approximate nearest neighbor search.

20.1 Key Idea

The key idea is to **split a high-dimensional vector into smaller subvectors** and quantize each independently.

20.2 PQ Algorithm

Given a vector $x \in \mathbb{R}^d$, we split it into m subvectors:

$$x = [x_1, x_2, \dots, x_m]$$

Each subvector has dimension d/m .

For each subvector we create a separate codebook $C^{(j)}$ of size k .

20.3 Example

Suppose:

- vector dimension $d = 8$
- number of subvectors $m = 4$

We split the vector:

$$x = [x_1 \ x_2 \mid x_3 \ x_4 \mid x_5 \ x_6 \mid x_7 \ x_8]$$

Each subvector is quantized using its own codebook.

20.4 PQ Representation

Instead of storing the original vector, we store the indices of the nearest codebook entries.

Listing 20.1: Product Quantization Encoding

```
Original vector:
[0.2 0.9 | -0.1 0.7 | 0.5 0.2 | -0.3 0.6]

Compressed form:
[3, 7, 1, 4]

Each number represents a codebook index.
```

20.5 Reconstruction

To reconstruct the vector:

$$\hat{x} = [c_{i_1}^{(1)}, c_{i_2}^{(2)}, \dots, c_{i_m}^{(m)}]$$

Where $c_{i_j}^{(j)}$ is the codebook vector from the j -th subcodebook.

20.6 Advantages of PQ

Product Quantization provides several benefits:

- **Efficient compression** — High-dimensional vectors can be compressed dramatically.
- **Fast similarity search** — Distances between vectors can be approximated efficiently using precomputed lookup tables.

20.7 Where PQ Is Used

Product Quantization is widely used in:

- FAISS (Facebook AI Similarity Search)
- Milvus
- Weaviate
- ScaNN (Google)

These systems use PQ to compress embedding vectors while still supporting fast similarity search.

20.8 PQ Parameters

Parameter	Description	Typical Values
d	Vector dimension	64-4096
m	Number of subvectors	4-64
k	Codebook size per subvector	256 (8 bits)
b	Bits per subvector	$\log_2(k)$

20.9 Memory Analysis

For a vector of dimension $d = 1024$ with FP32 precision:

- Original: $1024 \times 32 = 32,768$ bits

With PQ: $m = 32$ subvectors, $k = 256$ (8 bits):

- Compressed: $32 \times 8 = 256$ bits
- Compression ratio: $128\times$

21 Residual Quantization (RQ)

Product Quantization compresses vectors by splitting them into subvectors.

Residual Quantization takes a different approach: it compresses vectors iteratively by quantizing the remaining error.

21.1 Basic Idea

Instead of approximating a vector with a single codebook entry, residual quantization represents it as a sum of multiple codebook vectors:

$$x \approx c_1 + c_2 + \dots + c_k$$

Where each c_i comes from a different codebook.

21.2 Algorithm

Listing 21.1: Residual Quantization Algorithm

```
Input: Vector x, number of residuals R
Output: Codebook indices i1, i2, ..., iR

Initialize residual r = x

For t = 1 to R:
    Find nearest codebook vector:
        c_t = argmin_{c in C_t} ||r - c||
    Store index i_t
    Update residual:
        r = r - c_t

Return [i1, i2, ..., iR]
```

21.3 Step-by-Step Example

Step 1 — First quantization

- Original vector: $x = [0.8, 0.3]$
- Nearest codebook vector: $c_1 = [0.7, 0.2]$
- Residual: $r_1 = x - c_1 = [0.1, 0.1]$

Step 2 — Second quantization

- Nearest codebook vector: $c_2 = [0.1, 0.1]$
- Residual: $r_2 = r_1 - c_2 = [0.0, 0.0]$

Final reconstruction:

$$\hat{x} = c_1 + c_2 = [0.8, 0.3]$$

21.4 Advantages of Residual Quantization

Residual quantization offers several benefits:

- **Higher accuracy** — Multiple codebooks allow better approximation.
- **Progressive compression** — Residual levels can be truncated depending on memory constraints.
- **Flexible representation** — Vectors can be reconstructed with different levels of precision.

21.5 RQ vs PQ: Memory Comparison

For a vector of dimension $d = 1024$:

Method	Storage	Compression Ratio
Original (FP32)	32,768 bits	1×
PQ (m=32, 8 bits)	256 bits	128×
RQ (R=4, 8 bits)	32 bits	1024×

Note: RQ storage is $R \times \log_2(k)$ bits per vector, independent of dimension.

22 Product Quantization vs Residual Quantization

Both techniques compress high-dimensional vectors but use different strategies.

22.1 Comparison Table

Aspect	Product Quantization	Residual Quantization
Idea	Split vector into parts	Iterative error correction
Strength	Efficient ANN search	Higher reconstruction accuracy
Storage	$m \times \log_2(k)$ bits	$R \times \log_2(k)$ bits
Independence	Subvectors independent	Residuals dependent
Search speed	Very fast (lookup tables)	Slower (needs recomputation)
Typical use	Vector databases, embeddings	Image compression, audio

22.2 When to Use Each

Use **Product Quantization** when:

- Fast similarity search is required
- Vectors have moderate dimensionality (64-4096)
- You need predictable compression ratio

Use **Residual Quantization** when:

- High reconstruction accuracy is critical
- Progressive compression is needed
- Dimension is very high (e.g., 10K+)

22.3 Combined Approaches

Many modern systems combine these techniques:

- **PQ+RQ**: Apply PQ first, then residual quantize the errors
- **Optimized Product Quantization (OPQ)**: Apply rotation before PQ to improve variance distribution
- **Multi-codebook methods**: Combine multiple residual levels with product decomposition

Part VI

TurboQuant

23 TurboQuant Algorithm

23.1 Introduction

As we have seen throughout this book, the KV cache is one of the most significant memory bottlenecks in modern LLM inference. For long-context models with 32K, 128K, or even 1M tokens, the KV cache can consume hundreds of gigabytes per request — far exceeding the memory required for model weights themselves.

Traditional approaches to KV cache compression fall into two categories:

1. **Scalar quantization** (INT8, INT4) — treats each dimension independently, ignoring correlations between dimensions. While simple and fast, it struggles with outlier values and achieves limited compression ratios.
2. **Architectural changes** (GQA, MQA) — reduces the number of KV heads but requires model retraining and may reduce model quality.

Neither approach fully addresses the fundamental challenge: KV vectors are high-dimensional ($d = 4096$ or more) with strong correlations across dimensions, yet standard compression methods treat them as independent scalars.

23.1.1 A New Direction: Vector Quantization for KV Cache

TurboQuant is a recent technique proposed by researchers at Google that applies **vector quantization principles specifically to KV cache compression**. Unlike traditional vector quantization methods (such as Product Quantization) designed for static embeddings, TurboQuant is optimized for the unique characteristics of KV vectors:

- KV vectors are **dynamic** — they vary with input tokens and attention patterns
- KV vectors have **structured outliers** — certain dimensions consistently have larger magnitudes
- KV vectors require **fast reconstruction** — decoding must happen in real-time during inference
- KV vectors have **high intrinsic dimensionality** — simple scalar quantization loses important structure

TurboQuant addresses these challenges by combining four key techniques:

1. **Random rotations** — spread outlier information across dimensions
2. **Coarse scalar quantization** — provides a fast baseline approximation
3. **Random projection sketches** — compresses the residual error efficiently
4. **Sign compression** — achieves extreme compression of the sketch

The result is a vector quantization method that compresses KV vectors to 3–4 bits per dimension while maintaining attention accuracy comparable to FP16 — significantly better than scalar INT4 quantization at the same bit rate.

23.2 Core Idea

TurboQuant combines four key techniques to achieve 3–4 bits per dimension for KV vectors while maintaining high attention accuracy:

23.2.1 Random Rotation

The rotation $y = Rx$ spreads information across dimensions, eliminating outliers that hinder scalar quantization.

23.2.2 Coarse Scalar Quantization

The rotated vector is quantized coarsely (e.g., INT4) to produce $\tilde{y}$.

23.2.3 Residual Computation

The residual $r = y - \tilde{y}$ captures the quantization error.

23.2.4 Random Projection Sketch

The residual is projected to lower dimension: $z = Sr$, where S is a random projection matrix.

23.2.5 Sign Compression

Only the sign of each projected value is stored: $\text{sign}(z)$.

23.2.6 Reconstruction

The residual is approximated from the sign sketch: $\hat{r} = \alpha S^T \text{sign}(z)$, and the vector is reconstructed as $\hat{y} = \tilde{y} + \hat{r}$, then inverse rotated.

23.3 Numerical Example

Let's walk through TurboQuant with a concrete example.

23.3.1 Setup

Consider a 4-dimensional KV vector:

$$x = [0.83, -0.31, 0.27, 0.11]$$

23.3.2 Step 1: Random Rotation

For this example, we use the following random orthogonal rotation matrix (scaled Hadamard matrix):

$$R = \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

Applying R to x :

$$\begin{aligned} y_1 &= \frac{1}{2}(0.83 - 0.31 + 0.27 + 0.11) = 0.45 \\ y_2 &= \frac{1}{2}(0.83 + 0.31 + 0.27 - 0.11) = 0.65 \\ y_3 &= \frac{1}{2}(0.83 - 0.31 - 0.27 - 0.11) = 0.07 \\ y_4 &= \frac{1}{2}(0.83 + 0.31 - 0.27 + 0.11) = 0.49 \end{aligned}$$

Thus:

$$y = [0.45, 0.65, 0.07, 0.49]$$

23.3.3 Step 2: Coarse Scalar Quantization

Using scale factor $s = 0.25$:

First, compute y/s :

$$y/s = [1.8, 2.6, 0.28, 1.96]$$

Round to nearest integer:

$$\text{round}(y/s) = [2, 3, 0, 2]$$

Then reconstruct:

$$\tilde{y} = s \cdot \text{round}(y/s) = [0.50, 0.75, 0, 0.50]$$

23.3.4 Step 3: Residual Computation

$$r = y - \tilde{y} = [0.45 - 0.50, 0.65 - 0.75, 0.07 - 0, 0.49 - 0.50]$$

$$r = [-0.05, -0.10, 0.07, -0.01]$$

23.3.5 Step 4: Random Projection Sketch

Project to 2 dimensions using projection matrix S :

$$S = \begin{bmatrix} 0.5 & -0.5 & 0.5 & 0.5 \\ 0.5 & 0.5 & -0.5 & 0.5 \end{bmatrix}$$

Compute $z = Sr$:

$$\begin{aligned} z_1 &= 0.5(-0.05) - 0.5(-0.10) + 0.5(0.07) + 0.5(-0.01) \\ &= -0.025 + 0.05 + 0.035 - 0.005 = 0.055 \end{aligned}$$

$$\begin{aligned} z_2 &= 0.5(-0.05) + 0.5(-0.10) - 0.5(0.07) + 0.5(-0.01) \\ &= -0.025 - 0.05 - 0.035 - 0.005 = -0.115 \end{aligned}$$

Thus:

$$z = [0.055, -0.115]$$

23.3.6 Step 5: Sign Compression

$$\text{sign}(z) = [1, -1]$$

23.3.7 Step 6: Storage

Stored values:

- Quantized vector: $\tilde{y} = [0.50, 0.75, 0, 0.50]$
- Scale factor: $s = 0.25$
- Sign sketch: $\text{sign}(z) = [1, -1]$

23.3.8 Step 7: Reconstruction

First, reconstruct residual from sign sketch using $\alpha = 0.5$:

$$\hat{r} = \alpha S^T \text{sign}(z) = 0.5 \cdot \begin{bmatrix} 0.5 & 0.5 \\ -0.5 & 0.5 \\ 0.5 & -0.5 \\ 0.5 & 0.5 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Compute $S^T \text{sign}(z)$:

$$\begin{bmatrix} 0.5(1) + 0.5(-1) \\ -0.5(1) + 0.5(-1) \\ 0.5(1) + (-0.5)(-1) \\ 0.5(1) + 0.5(-1) \end{bmatrix} = \begin{bmatrix} 0.5 - 0.5 \\ -0.5 - 0.5 \\ 0.5 + 0.5 \\ 0.5 - 0.5 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \\ 1 \\ 0 \end{bmatrix}$$

Multiply by $\alpha = 0.5$:

$$\hat{r} = [0, -0.5, 0.5, 0]$$

Now reconstruct $\hat{y}$:

$$\hat{y} = \tilde{y} + \hat{r} = [0.50 + 0, 0.75 - 0.5, 0 + 0.5, 0.50 + 0] = [0.50, 0.25, 0.50, 0.50]$$

Finally, apply inverse rotation $\hat{x} = R^{-1}\hat{y}$. Since R is orthogonal, $R^{-1} = R^T$:

$$\hat{x} = R^T \hat{y} = \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0.50 \\ 0.25 \\ 0.50 \\ 0.50 \end{bmatrix}$$

Compute each component:

$$\begin{aligned} \hat{x}_1 &= \frac{1}{2}(0.50 + 0.25 + 0.50 + 0.50) = \frac{1}{2}(1.75) = 0.875 \\ \hat{x}_2 &= \frac{1}{2}(0.50 - 0.25 + 0.50 - 0.50) = \frac{1}{2}(0.25) = 0.125 \\ \hat{x}_3 &= \frac{1}{2}(0.50 + 0.25 - 0.50 - 0.50) = \frac{1}{2}(-0.25) = -0.125 \\ \hat{x}_4 &= \frac{1}{2}(0.50 - 0.25 - 0.50 + 0.50) = \frac{1}{2}(0.25) = 0.125 \end{aligned}$$

Thus:

$$\hat{x} = [0.875, 0.125, -0.125, 0.125]$$

23.3.9 Results

Dimension	Original x	Reconstructed $\hat{x}$	Error
1	0.83	0.875	0.045
2	-0.31	0.125	0.435
3	0.27	-0.125	0.395
4	0.11	0.125	0.015

Note: The reconstruction error is higher in this simplified example because we used a very small dimension ($d = 4$) and aggressive compression ($m = 2$). In practice, with $d = 4096$ and $m = 32$, the error is much smaller (typically < 0.01 per dimension).

Part VII

Quantization in Production ML Systems

24 Quantization in Modern Inference Engines

Production LLM systems rely on specialized inference engines designed for high throughput and low latency.

These engines integrate quantization into the model loading and execution pipeline.

Two widely used systems are:

- **vLLM**
- **TensorRT-LLM**

24.1 Quantization in vLLM

vLLM is an inference engine designed for large-scale LLM serving.

Its key innovations include:

- Paged Attention
- efficient KV cache management
- continuous batching
- high-throughput GPU execution

24.1.1 vLLM Inference Pipeline

A simplified pipeline looks like:

Listing 24.1: vLLM Inference Pipeline



Quantization affects several parts of this pipeline.

24.1.2 Weight Quantization in vLLM

vLLM supports quantized models produced using methods such as:

- GPTQ
- AWQ

- bitsandbytes INT8 / INT4

When loading a model, weights are stored in compressed integer formats, reducing GPU memory usage.

24.1.3 KV Cache Optimization in vLLM

vLLM primarily improves KV cache efficiency through **Paged Attention**, which organizes memory into pages rather than large contiguous allocations.

This improves:

- GPU memory utilization
- multi-request batching
- system throughput

Future work may integrate KV compression techniques like TurboQuant directly into inference engines.

24.2 Quantization in TensorRT-LLM

TensorRT-LLM is NVIDIA's optimized inference framework for transformer models.

Unlike vLLM, which focuses on runtime scheduling and memory management, TensorRT-LLM focuses on **kernel-level optimization** and **hardware acceleration**.

24.2.1 TensorRT-LLM Deployment Pipeline

Listing 24.2: TensorRT-LLM Pipeline

```
PyTorch / HuggingFace model
    |
    v
TensorRT conversion
    |
    v
Calibration
    |
    v
Kernel optimization
    |
    v
Quantized inference engine
```

During the conversion stage, TensorRT generates optimized kernels for the target GPU architecture.

24.2.2 INT8 and FP8 Optimization

TensorRT-LLM supports multiple precision formats:

Precision	Use Case
FP16	standard inference
INT8	high-performance inference
FP8	emerging format for training and inference

These formats allow models to execute efficiently on GPU Tensor Cores.

24.3 Comparison of Inference Engines

Feature	vLLM	TensorRT-LLM
Primary focus	Memory management	Kernel optimization
KV cache	Paged Attention	Optimized kernels
Weight quantization	GPTQ, AWQ, bitsandbytes	INT8, FP8
Deployment	Flexible, Python-first	NVIDIA-optimized
Hardware support	Multi-GPU	NVIDIA GPUs only

25 Hardware Support for Quantization

Low-precision computation is heavily supported by modern AI hardware.

Many accelerators now include specialized instructions for:

- INT8 matrix multiplication
- FP16 tensor operations
- FP8 mixed-precision computation

These capabilities enable quantized models to run significantly faster than full-precision models.

25.1 GPU Tensor Cores

Modern GPUs include **Tensor Cores**, specialized units designed for matrix multiplication used in deep learning.

Tensor Cores accelerate operations such as:

$$C = A \times B$$

Where matrices can use mixed precision formats such as:

- FP16
- BF16
- INT8
- FP8

These operations provide extremely high throughput compared to standard GPU compute units.

25.2 Tensor Core Throughput Comparison

Precision	Relative Throughput	Typical Use
FP32	1×	Baseline
FP16/BF16	2× to 4×	Mixed precision training
INT8	4× to 8×	Inference optimization
FP8	8× to 16×	Next-gen training/inference

25.3 INT8 Inference Kernels

Many production systems rely on **INT8 inference kernels**.

25.3.1 Advantages

- faster matrix multiplication
- lower memory bandwidth usage
- improved energy efficiency

25.3.2 Applications

INT8 kernels are widely used for:

- computer vision models
- recommendation systems
- transformer inference pipelines

Frameworks such as TensorRT automatically generate optimized kernels for INT8 inference.

25.4 FP8 Training Hardware

Recent GPU architectures have introduced FP8 support, which reduces memory requirements while maintaining adequate numerical range.

25.4.1 Benefits of FP8

- faster training
- reduced memory footprint
- improved throughput

Hardware such as **NVIDIA H100** GPUs supports FP8 operations for large-scale model training.

25.5 Quantization Support Across Hardware

Hardware	INT8	FP8	Key Feature
NVIDIA A100	Yes	No	Tensor Cores, INT8 acceleration
NVIDIA H100	Yes	Yes	FP8 Tensor Cores, Transformer Engine
AMD MI300	Yes	No	Matrix Cores
Google TPU	Yes	No	MXU (Matrix Multiply Unit)

26 Trade-offs in Quantization

Although quantization provides significant benefits, it also introduces trade-offs that engineers must consider.

These trade-offs involve balancing:

- model accuracy
- compression ratio
- inference latency
- system complexity

26.1 Accuracy vs Compression

Lower precision formats provide stronger compression but increase quantization error.

Format	Compression	Accuracy Impact
FP16	1× (baseline)	None
INT8	2×	Minimal
INT4	4×	Moderate
TurboQuant	~ 4× to 5×	Low to moderate

However, aggressive compression can degrade model performance, especially for smaller models or sensitive tasks.

26.2 Latency vs Decompression Overhead

Some quantization techniques introduce additional computation for reconstruction.

For example:

- vector quantization
- residual sketches
- adaptive precision

These methods must be carefully integrated into inference kernels to avoid increasing latency.

26.3 Calibration Complexity

Post-training quantization methods often require calibration datasets to estimate scale factors.

Calibration introduces additional complexity into deployment pipelines:

- representative data must be selected
- scale factors must be computed per layer
- validation ensures accuracy

26.4 Trade-off Summary

Method	Compression	Accuracy	Complexity
FP16	Low	High	Low
INT8	Medium	High	Low
INT4	High	Medium	Medium
GPTQ/AWQ	High	Medium-High	Medium
TurboQuant	Very High	Medium	High

27 Choosing a Quantization Strategy

The optimal quantization strategy depends on deployment constraints.

27.1 Decision Framework

Scenario	Recommended Approach
Edge devices (mobile, embedded)	INT8 / INT4 quantization with lightweight kernels
Cloud inference	GPTQ / AWQ 4-bit weight quantization
Large-scale serving	Mixed precision + KV cache optimization
Long-context models	KV compression (TurboQuant, vector quantization)
Fine-tuning with limited memory	QLoRA (4-bit base + LoRA adapters)
Low-latency requirements	FP8/INT8 with optimized kernels

27.2 Combined Strategies

Production systems often combine multiple techniques:

Listing 27.1: Typical Production Quantization Stack

```
Model Weights:
  GPTQ 4-bit quantization
  |
  v
KV Cache:
  TurboQuant or INT8 compression
  |
  v
Inference Engine:
  vLLM with Paged Attention
  |
  v
Hardware:
  NVIDIA H100 with FP8 kernels
```

This layered approach maximizes both memory efficiency and inference throughput.

Part VIII

Future Directions in Quantization

28 Adaptive Precision

Traditional quantization applies the same numerical precision across an entire tensor or model.
For example:

Method	Precision
INT8 quantization	8 bits for all weights
INT4 quantization	4 bits for all weights

However, different parts of a neural network have different sensitivity to numerical noise. Some layers are highly sensitive to precision loss, while others are extremely robust.

28.1 Layer-Wise Precision

Adaptive precision addresses this problem by assigning different precision levels to different parts of the model.

Critical layers may use higher precision:

- Attention layers $\rightarrow$ FP16
- Feed-forward layers $\rightarrow$ INT8
- Embedding layers $\rightarrow$ INT4

This approach allows models to maintain accuracy while still achieving significant compression.

28.2 Token-Level Precision

Another promising direction is dynamic precision during inference.

For example:

- early tokens may require higher precision
- later tokens may tolerate aggressive compression

This allows the system to adapt quantization dynamically during generation.

28.3 Benefits of Adaptive Precision

- **Higher overall accuracy** — sensitive components retain precision
- **Better compression** — robust components compressed aggressively
- **Flexible deployment** — precision can adapt to hardware constraints

29 Learned Quantization

Most quantization techniques rely on fixed mathematical mappings, such as linear scaling.

However, neural networks may benefit from learning the quantization function itself.

29.1 Core Idea

In **learned quantization**, the model jointly optimizes:

- network weights
- quantization parameters (scale, zero-point, clipping ranges)

These methods allow models to adapt to quantization noise during training.

29.2 Differentiable Quantization

A common technique is to approximate the quantization function with a differentiable surrogate during training.

Forward pass:

$$x_q = \text{quantize}(x)$$

Backward pass (Straight-Through Estimator):

$$\frac{\partial L}{\partial x} \approx \frac{\partial L}{\partial x_q}$$

This allows gradient-based optimization despite the discrete quantization operation.

29.3 Learned Parameters

Quantization parameters that can be learned include:

- **Scale factors** — s in $q = \text{round}(x/s)$
- **Zero-point offsets** — z in asymmetric quantization
- **Clipping ranges** — α in $x = \text{clip}(x, -\alpha, \alpha)$
- **Quantization grids** — non-uniform spacing

29.4 Benefits of Learned Quantization

- **Higher accuracy** — parameters optimized for the specific model
- **Better compression** — can achieve lower precision with same accuracy
- **Task-aware** — quantization adapted to end-task objective

30 Token-Level KV Compression

One of the biggest challenges in modern LLM systems is long-context inference.

As we saw earlier, KV cache memory grows linearly with sequence length.

30.1 Motivation

For a 128K token context, KV cache can exceed 300 GB even with INT4 compression. Future systems must go beyond tensor-level compression.

30.2 Token-Level Strategies

Future systems may compress KV states at a **token-level granularity** rather than compressing entire tensors.

Possible strategies include:

- **Compressing older tokens** — earlier tokens compressed more aggressively
- **Selective pruning** — removing tokens with low attention scores
- **Adaptive precision** — higher precision for recent tokens
- **Token merging** — combining similar tokens into representatives

30.3 Token-Level KV Compression Example

Listing 30.1: Token-Level Adaptive KV Compression

```
Token 1 (old):  INT2 compression (very aggressive)
Token 2 (old):  INT2 compression
Token 3 (medium): INT4 compression
Token 4 (medium): INT4 compression
Token 5 (recent): INT8 compression
Token 6 (current): FP16 (no compression)
```

30.4 Expected Benefits

- **Linear to sublinear memory growth** — memory may grow slower than sequence length
- **Million-token contexts** — becomes feasible on single GPUs
- **Better user experience** — longer conversations without memory limits

31 Hybrid Compression Methods

Another promising direction is combining multiple compression techniques.

Instead of relying on a single strategy, future systems may combine:

- quantization
- low-rank decomposition
- sparsity
- vector compression

31.1 Example: Low-Rank + Quantization

Listing 31.1: Hybrid Compression Pipeline

```
Weight matrix
  |
  v
Low-rank decomposition:  $W = UV$ 
  |
  v
Quantize low-rank factors:  $U_q, V_q$ 
  |
  v
Store compressed representation
```

31.2 Combined Approach Benefits

Technique	Compression	Limitation
Quantization only	4× to 8×	Accuracy loss at extreme compression
Low-rank only	Variable	Limited by rank
Hybrid	16× to 32×	May preserve accuracy better

31.3 Other Hybrid Combinations

- **Quantization + Pruning** — remove unimportant weights, then quantize
- **Sparse Quantization** — maintain sparse structure in compressed format
- **Quantized LoRA** — quantize base model, fine-tune with low-rank adapters (QLoRA)

32 Hardware–Algorithm Co-Design

Future quantization research will likely involve closer integration between hardware design and model algorithms.

Historically, numerical formats were defined primarily by hardware vendors.

Today, the relationship is becoming more collaborative.

32.1 Examples of Co-Design

- **FP8 formats** — designed specifically for AI workloads (E4M3, E5M2)
- **Specialized tensor cores** — optimized for INT8 and BF16 operations
- **Custom accelerators** — designed for low-precision inference (e.g., Groq, Cerebras)

32.2 Benefits of Co-Design

- **Higher efficiency** — hardware matched to algorithmic needs
- **New capabilities** — enabling formats not previously feasible
- **Reduced overhead** — less software workarounds for hardware limitations

32.3 The Co-Design Feedback Loop

Listing 32.1: Hardware-Algorithm Co-Design Loop

```
Algorithm Needs
    |
    v
Hardware Design
    |
    v
New Capabilities
    |
    v
Improved Algorithms
    |
    v
(cycle repeats)
```

33 Toward Efficient AI Systems

As AI models continue to scale, efficiency will become just as important as model capability. Future AI systems will likely combine several complementary techniques:

- **Quantization** — reduce numerical precision
- **Low-rank methods** — decompose large matrices
- **Sparse computation** — skip zero-valued operations
- **Hardware acceleration** — specialized compute units

33.1 The Efficiency Stack

Layer	Techniques
Algorithm	Adaptive precision, learned quantization, hybrid methods
Model Architecture	GQA, MQA, mixture-of-experts
System Software	Paged attention, continuous batching
Hardware	Tensor cores, FP8 units, custom accelerators

33.2 Expected Outcomes

Together, these techniques will enable:

- **Larger models** — billion-parameter models on edge devices
- **Longer context windows** — million-token contexts on single GPUs
- **Lower deployment costs** — reduced hardware requirements
- **Higher system throughput** — more requests per second

34 Final Thoughts

Quantization has transformed from a niche optimization into a fundamental building block of modern AI systems.

From scalar quantization to vector compression methods such as TurboQuant, these techniques enable large models to run efficiently on real-world hardware.

34.1 The Journey

- **Early days** — FP32 was the default, quantization was exotic
- **Recent years** — INT8, FP16, BF16 became mainstream
- **Today** — INT4, GPTQ, AWQ enable billion-parameter models on consumer GPUs
- **Future** — Adaptive precision, hybrid methods, hardware co-design

34.2 Key Takeaways

1. Quantization is essential for scaling AI systems
2. Different techniques suit different deployment scenarios
3. KV cache compression is critical for long-context models
4. Vector quantization enables compression beyond scalar methods
5. Production systems combine quantization with system-level optimizations
6. Future innovation lies at the intersection of algorithms and hardware

34.3 A Look Ahead

As models continue to grow in size and complexity, quantization will remain central to enabling scalable AI systems.

Future innovations will likely come from the intersection of:

- **algorithm design** — new compression techniques
- **hardware architecture** — specialized accelerators
- **distributed systems engineering** — efficient serving infrastructure

Understanding these techniques will therefore be essential for anyone building the next generation of AI infrastructure.

Quantization is not just about saving memory. It is about making AI systems scalable, efficient, and accessible.
--